LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 1
Student Name: P. Santosh Kumar
Roll No.: A24126552082
Date: 30-08-2026

TITLE

Student Profile Manager Using HTML, CSS and JavaScript

AIM

To develop a **Student Profile Manager** using HTML, CSS, and JavaScript that accepts student details and dynamically generates a student profile using JavaScript classes and DOM manipulation.

OBJECTIVES

- To understand the concept of JavaScript classes and objects.
- To create a Student class for storing student information.
- To accept student details through HTML input fields.
- To use DOM manipulation for dynamically displaying the student profile.
- To handle button click events using event listeners.
- To apply CSS styling to create an attractive and responsive webpage.

CONCEPTS COVERED

- HTML Forms
- CSS Styling
- JavaScript Classes
- JavaScript Objects
- Constructor
- DOM Manipulation
- Event Listeners

- Input Validation
- Dynamic HTML Elements
- Template-free DOM Creation
- Responsive Web Design

TASK DESCRIPTION

Create a webpage titled **"Student Profile Manager"**.

The application should:

- Provide input fields for:
    - Name
    - Roll Number
    - Department
    - CGPA
- Provide a **Generate Profile** button.
- Create a Student class with the properties:
    - Name
    - Roll Number
    - Department
    - CGPA
- Create a Student object using the values entered by the user.
- Display the student details dynamically on the webpage using DOM manipulation.
- Display an alert message if any required field is left empty.
- Format the CGPA to two decimal places.
- Apply CSS styling to the form and generated profile.
- Make the webpage responsive for smaller screen sizes.

ALGORITHM / PROCEDURE

1. Create an HTML document with the required input fields.
2. Add input fields for Name, Roll Number, Department, and CGPA.
3. Add a **Generate Profile** button.

4. Create a CSS file to style the webpage, form, button, and profile card.
5. Create a JavaScript Student class.
6. Define a constructor to initialize the student properties.
7. Access the HTML elements using getElementById().
8. Add a click event listener to the Generate Profile button.
9. Read the values entered by the user.
10. Remove unnecessary spaces using trim().
11. Check whether all required fields contain values.
12. Display an alert if any field is empty.
13. Create a Student object using the entered details.
14. Clear any previously generated profile.
15. Dynamically create the profile card using DOM methods.
16. Display the student's Name, Roll Number, Department, and CGPA.
17. Append the generated profile to the webpage.
18. Test the application using different student details.

PROGRAM / SOURCE CODE

The following pages contain the submitted HTML source code screenshots.

Index.html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Student Profile Manager</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <div class="page-wrap">
      <h1>Student Profile Manager</h1>

      <div class="student-form">
        <label for="name">Name</label>
        <input id="name" type="text" placeholder="Enter student name" value="Ravi Kumar" />

        <label for="rollNumber">Roll Number</label>
        <input id="rollNumber" type="text" placeholder="Enter roll number" value="A23126552001" />

        <label for="department">Department</label>
        <input id="department" type="text" placeholder="Enter department" value="CSE (AI & ML)" />

        <label for="cgpa">CGPA</label>
        <input id="cgpa" type="number" step="0.01" min="0" max="10" placeholder="Enter CGPA" value="8.42" />

        <button id="generateBtn" type="button">Generate Profile</button>
      </div>

      <div id="profileContainer" aria-live="polite"></div>
    </div>
      You, 2 weeks ago • renamed …
    <script src="script.js"></script>
  </body>
</html>
```

Styles.css

```css
.student-form label {
  font-weight: 600;
  color: #1f2937;
}

.student-form input {
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  font-size: 1rem;
}

.student-form input:focus {
  outline: 2px solid #60a5fa;
  border-color: #60a5fa;
}

button {
  margin-top: 12px;
  padding: 12px 18px;
  border: none;
  border-radius: 12px;
  background: linear-gradient(90deg, #2563eb, #7c3aed);
  color: #fff;
  font-weight: 600;
  cursor: pointer;
  transition: transform 0.2s ease;
}

button:hover {
  transform: translateY(-1px);
}
```

```css
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #eef4ff, #f8f4ff);
  color: #1f2937;
}
      You, 2 weeks ago • renamed …
.page-wrap {
  max-width: 700px;
  margin: 40px auto;
  padding: 30px 24px;
}

h1 {
  text-align: center;
  margin-bottom: 24px;
  color: #1e3a8a;
}

.student-form {
  background: #ffffff;
  padding: 24px;
  border-radius: 16px;
  box-shadow: 0 8px 24px rgba(37, 99, 235, 0.12);
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

```css
.label {
  font-weight: 700;
  color: #334155;
}

.value {
  color: #0f172a;
  text-align: right;
}

@media (max-width: 600px) {
  .page-wrap {
    padding: 20px 16px;
  }

  .detail-row {
    flex-direction: column;
    align-items: flex-start;
    gap: 4px;
  }

  .value {
    text-align: left;
  }
}
```

Script.js

```
class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = Number(cgpa).toFixed(2);
  }
}

const generateBtn = document.getElementById('generateBtn');
const profileContainer = document.getElementById('profileContainer');

generateBtn.addEventListener('click', () => {
  const name = document.getElementById('name').value.trim();
  const rollNumber = document.getElementById('rollNumber').value.trim();
  const department = document.getElementById('department').value.trim();
  const cgpa = document.getElementById('cgpa').value.trim();

  if (!name || !rollNumber || !department || !cgpa) {
    alert('Please fill in all student details.');
    return;
  }       You, 2 weeks ago • renamed …

  const student = new Student(name, rollNumber, department, cgpa);
  profileContainer.innerHTML = '';

  const profileCard = document.createElement('div');
  profileCard.className = 'profile-card';

  const heading = document.createElement('h2');
  heading.textContent = 'Student Profile';
  profileCard.appendChild(heading);
```

```
  const details = [
    ['Name', student.name],
    ['Roll No', student.rollNumber],
    ['Department', student.department],
    ['CGPA', student.cgpa]
  ];

  details.forEach(([label, value]) => {
    const row = document.createElement('div');
    row.className = 'detail-row';

    const labelEl = document.createElement('span');
    labelEl.className = 'label';
    labelEl.textContent = label + ' :';

    const valueEl = document.createElement('span');
    valueEl.className = 'value';
    valueEl.textContent = value;

    row.appendChild(labelEl);
    row.appendChild(valueEl);
    profileCard.appendChild(row);
  });

  profileContainer.appendChild(profileCard);
});

generateBtn.click();
```

CODE EXPLANATION

Component	Purpose
class Student	Creates a Student class.
constructor()	Initializes student details
this.name	Stores the name.
this.rollNumber	Stores the roll number.
this.department	Stores the department.
this.cgpa	Stores CGPA up to 2 decimal places.
getElementById()	Accesses HTML elements.
addEventListener()	Handles the button click.
.value	Gets input values.
.trim()	Removes extra spaces.
alert()	Shows an error message.
textContent	textContent
appendChild()	Adds elements to the webpage.
innerHTML = ''	Clears the old profile.
:hover	Adds an effect when the mouse is over the button.
@media	Makes the page responsive.

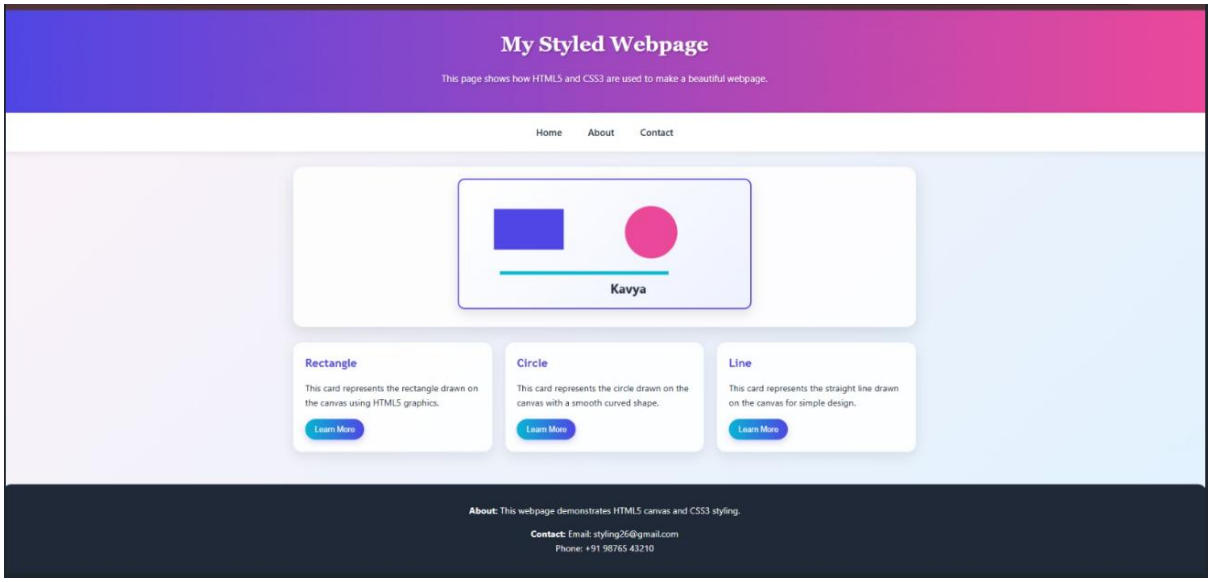
TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Observed Result	Status
TC-01	Open the webpage	Student Profile Manager should be displayed	Page displayed correctly	Pass
TC-02	Enter valid student details	Student profile should be generated	Profile generated correctly	Pass
TC-03	Leave Name field empty	Alert should be displayed	Alert displayed	Pass
TC-04	Leave Roll Number field empty	Alert should be displayed	Alert displayed	Pass
TC-05	Enter CGPA as 8.42	CGPA should be displayed with two decimal places	8.42 displayed	Pass
TC-06	Enter different student details	New student profile should be generated	Profile updated correctly	Pass

TC-07	Click Generate Profile again	Previous profile should be replaced	Profile replaced correctly	Pass
TC-08	Open webpage on smaller screen	Layout should adjust to screen size	Responsive layout displayed	Pass

---

OUTPUT:



RESULT

Thus, the Student Profile Manager was successfully developed using HTML, CSS, and JavaScript. A JavaScript Student class was used to create student objects, and DOM manipulation was used to dynamically generate and display the student profile. Input validation, event handling, CSS styling, and responsive design were also successfully implemented.